

PARALLEL COMPUTING — CLASS 0 DIAGNOSTIC

Task1

x = 5

y = 2

x = x + y # x = 7, y = 2

y = x * 2 # x = 7, y = 14

x = y - x # x = 7, y = 14

x=7, y=14

Task2

numbers = [10, 20, 30, 40, 50]

total = 0

for i in range(len(numbers)):

total = numbers[i]

print(total)

I guess task to find sum of numbers, and bug is in fourth line

It actually only saves the **last value**

total = numbers[i] must be total += numbers[i]

Task 3:

a) Outer loop runs 10 times and inner loop runs 10 times: 100 times.

b) If both loops run from 0 to 999, each loop runs 1000 times: 1,000,000 times.

c) Outer loop = 10 times, inner loop = 1,000,000 times: 10,000,000 times.

Task 4

Algorithm A is $O(n)$, and Algorithm B is $O(n^2)$. Algorithm B grows much faster. For $n = 1,000,000$, A does about 1 million operations, while B does about 1 trillion.

Task 5

$2^n > n^2 > n \log_2 n > n > \log_2 n > 1$.

2^n is the most problematic, and n^2 can also become very slow for large n .

Task 6

If the list is not known to be sorted, I use linear search and check elements one by one until I find 45.

If the list is sorted, I use binary search and repeatedly remove half of the search area. Binary search is faster because its complexity is $O(\log n)$, while linear search is $O(n)$.

Task 7

A simple algorithm:

max_value = first_number

for number in data:

```
if number > max_value:
    max_value = number
```

Each number needs to be examined **once**, so the complexity is: $O(n)$

For 500 million numbers, that means about **500 million checks**.

We **do not need to store all 500 million numbers** at the same time. You can read them one by one or process them in chunks.

Task 8

The best data structure is a **dictionary / hash map**.

Example:

```
counts = {}
```

```
for item in data:
    if item in counts:
        counts[item] += 1
    else:
        counts[item] = 1
```

Step by step, the dictionary updates the count every time a value appears.

Task 9

For two **2×2 matrices**, there are **8 individual multiplication operations**.

Why? There are 4 output cells, and each requires 2 multiplications:

$$4 \times 2 = 8$$

Task 10

Even if the CPU is very fast, the program can still be slow because:

1. Memory is slower than CPU — the CPU may wait for data to arrive from RAM.
2. Cache misses — if the array is too large for CPU cache, data must be fetched from slower memory.
3. Memory bandwidth limitation — RAM can only transfer a certain amount of data per second.

Task 11

The operating system finds the program, loads it into memory, creates a process, and gives it CPU time. The CPU executes the instructions. The program may access memory, files, or other resources, then produces output. When it finishes, the OS releases its resources.

Task 12

A is CPU-bound, B is memory-bound, C is I/O-bound, and D mostly waits for user input. All four can run together because the operating system schedules them. With multiple cores, programs can run in parallel. With one CPU core, only one program executes at a time, and the OS quickly switches between them.

Task 13

I do not fully agree. A computer that is twice as fast does not always make the whole program twice as fast. Some parts may depend on disk, memory, network, or other bottlenecks. The time could become less than 10 hours, but not necessarily exactly 5 hours.

Task 14

Possible ways to make it faster:

- Use a better algorithm.
- Reduce unnecessary calculations.
- Avoid repeating the same work.
- Use better data structures.
- Use faster hardware.
- Use multiple CPU cores.
- Use parallel processing.
- Improve memory usage.
- Reduce unnecessary file or disk access.
- Cache data that is used many times.
- Optimize slow parts of the code.
- Process data in chunks if needed.

The main idea is: Do not optimize everything equally. Find what takes the most time first.

Task 15

After making A, B, D and E 10× faster, total time becomes about 50.9 seconds. The overall program is only about 1.16× faster. Most optimization effort should go to C because it takes 50 seconds and is the main bottleneck.

Task 16

Because marks are only from 0 to 100, we can use a very efficient approach: keep a frequency array of size 101.

```
counts = [0] * 101
```

```
total = 0
```

```
minimum = 101
```

```
maximum = -1
```

```
above_90 = 0
```

```
below_40 = 0
```

```
for mark in marks:
```

```
    counts[mark] += 1
```

```
    total += mark
```

```
    if mark < minimum:
```

```
        minimum = mark
```

```
    if mark > maximum:
```

```
        maximum = mark
```

```
    if mark >= 90:
```

```
        above_90 += 1
```

```
    if mark < 40:
```

```
        below_40 += 1
```

```
average=10,000,000 / total
```

For the median, we don't need to sort all 10 million values. We can use the counts array:

```
counts[0] → how many students got 0
```

```
counts[1] → how many got 1
```

```
...
```

```
counts[100] → how many got 100
```

Then we go through counts and find where the middle student falls.

Data structures

We need only:

- `counts[101]` — frequency of every possible mark
- `total`
- `minimum`
- `maximum`
- counters for ≥ 90 and < 40

So we do not need to keep all 10 million marks in memory.

How many times is the data examined?

The main dataset is examined only **once**:

$O(n)$

where:

$n = 10,000,000$

Then we scan only the 101-element frequency array to find the median, which is very small.

Memory requirements

The frequency array contains only 101 numbers, so memory usage is very low.

Instead of storing:

10,000,000 marks

we basically store:

101 counters

Likely bottleneck

The main bottleneck will probably be:

reading and processing the 10 million values.

Especially if the data comes from a file, disk I/O can take a significant amount of time.

How could we make it faster?

We could:

- process data in chunks
- use multiple CPU cores
- let each core calculate its own counts, min, max, and sum
- combine the partial results at the end
- avoid sorting all 10 million values